Entornos de Desarrollo

Desarrollo de un Programa Informático

SCV - Sistemas de Control de Versiones

SCV - Sistemas de Control de Versiones I

Herramientas que permite registrar, organizar y gestionar los cambios que se realizan en los archivos de un proyecto de software a lo largo del tiempo.



SCV - Sistemas de Control de Versiones I

Herramientas que permite registrar, organizar y gestionar los cambios que se realizan en los archivos de un proyecto de software a lo largo del tiempo.

Su objetivo principal es mantener un historial de modificaciones, de manera que los desarrolladores puedan saber qué cambios se han realizado, cuándo se hicieron y quién los realizó, además de poder recuperar versiones anteriores cuando sea necesario.



SCV - Sistemas de Control de Versiones I

Herramientas que permite registrar, organizar y gestionar los cambios que se realizan en los archivos de un proyecto de software a lo largo del tiempo.

Su objetivo principal es mantener un historial de modificaciones, de manera que los desarrolladores puedan saber qué cambios se han realizado, cuándo se hicieron y quién los realizó, además de poder recuperar versiones anteriores cuando sea necesario.

Son especialmente importantes en el desarrollo de software porque permiten trabajar de forma segura y colaborativa. Varios desarrolladores pueden modificar un mismo proyecto, mantener sus propios cambios y posteriormente integrarlos. Si dos personas modifican una misma parte del código de forma incompatible, el sistema puede detectar un conflicto, que deberá ser revisado y resuelto.



SCV - Sistemas de Control de Versiones II

- Registrar cambios e historial: almacena las modificaciones realizadas en el proyecto y su evolución.
- Recuperar versiones anteriores: facilita volver a un estado previo del proyecto.
- Trabajar en equipo: permite que varios desarrolladores trabajen sobre el mismo proyecto.
- Crear ramas (branches): permite desarrollar funcionalidades de forma independiente sin modificar directamente la versión principal.
- Fusionar cambios (merge): permite integrar modificaciones procedentes de diferentes ramas.
- Detectar conflictos: identifica cambios incompatibles que deben ser revisados manualmente.
- Identificar autores: permite conocer quién realizó cada modificación.



Git

Es uno de los sistemas de control de versiones más utilizados actualmente. Git es un sistema distribuido, lo que significa que cada desarrollador dispone de una copia del repositorio y de su historial de cambios. Plataformas como GitHub, GitLab o Bitbucket permiten alojar repositorios Git y facilitan la colaboración entre los miembros de un equipo.



Comandos git desde repositorio local



- git init
  - crea un repositorio git local.
- git config --global user.name "Nombre de usuario"
- git config --global user.email "correo@correo.com"
- git add .
  - Preparamos **todos** los archivos para hacer un commit.
- git commit -m "Primer commit."
  - Creamos el commit con su comentario.
- git remote add origin https://github.com/USUARIO/REPO.git
  - Para modificarlo git remote set-url origin
- git branch -M main
  - Elegimos la rama a la que subir los cambios
- git push -u origin main
  - Subimos los cambios. Tras el primer push no es necesario especificar -u origin main.

Comandos git clonando repositorio



- git config --global user.name "Nombre de usuario"
- git config --global user.email "correo@correo.com"
- git clone https://github.com/USUARIO/REPO.git
- git add .
- git commit -m “Comentario.”
- git push
- git pull
- git fetch
- git status

MarkDown (MD) I

Markdown es un lenguaje de marcado ligero que permite dar formato a un documento de texto plano utilizando símbolos especiales de manera que son fáciles de leer tanto en crudo como procesado. Permite:

- Escribir rápido: Permite aplicar estilos (como negritas, títulos o listas) sin separar las manos del teclado para usar el ratón
- Compatibilidad universal: Al estar guardado en un archivo de texto plano (generalmente con extensión .md), se puede abrir y leer en cualquier dispositivo o sistema operativo sin depender de un programa privativo o pesado como Microsoft Word.
- Exportación fácil: El contenido se puede transformar de forma automática a otros formatos como HTML, PDF o páginas web completas.

MarkDown (MD) II

Sintaxis básica:

- Títulos: Se usa el símbolo de almohadilla (#) al principio de la línea. Más almohadillas indican un título más pequeño (# para H1, ## para H2).
- Negrita: Se rodea el texto con dos asteriscos o guiones bajos, por ejemplo: **texto**.
- Cursiva: Se rodea el texto con un solo asterisco o guión bajo, por ejemplo: *texto*.
- Listas: Se usan asteriscos (*), guiones (-) o números (1.) al inicio de cada línea.
- Enlaces: Se escriben entre corchetes para el texto y paréntesis para la dirección, como [Texto del enlace](https://ejemplo.com)

Texto de ejemplo:

https://github.com/profesor-raul-lago/Profesor_LagoRaul/blob/main/Prueba.md

Ejercicio 01 I

Instalad Git en el ordenador (https://git-scm.com/install/windows) y cread un repositorio público en GitHub (https://github.com/). Mediante comandos subid al repositorio un archivo de markdown (.md) con la siguiente información:

- ¿Qué es un programa informático?
- Diferencia entre código fuente, código objeto y código ejecutable.
- Etapas del desarrollo del software.

Se debe incluir al menos una **imagen** enlazada al archivo .md y un **enlace al repositorio**. El texto del archivo .md debe estar formateado de manera coherente. Se puede incluir en el repositorio carpetas y archivos adicionales.



Ejercicio 01 II

Nomenclatura:

- Repositorio: CursoModalidad_Apellido/s_Nombre/s (1DAMP_LagoCastro_RaulAngel)
- Archivo .md: IntroduccionSoftware_Apellido/s_Nombre/s.md (IntroduccionSoftware_LagoCastro_RaulAngel.md)
- Archivos adicionales: usad nombres descriptivos, fasjdfpouisahjfoiusajh.jpg no es un nombre descriptivo.

Entrega:

- Se entregará el archivo .md en la tarea habilitada en Teams. Es necesario que el archivo .md incluya el enlace al repositorio (si no la entrega no contará como aprobada).

